

Teoria de Grafos

Grafos de Árvores

Árvores Geradoras Mínimas

Prof. Dr. Marcos W. Rodrigues

marcos.rodrigues@

Roteiro

- Grafos de Árvores
 - Distância entre pares de vértices
 - Excentricidade de vértice
 - Centro, Raio e Diâmetro de um grafo Árvore
- Árvores Geradoras
 - Galho e Corda de uma Árvore Geradora
 - Rank e Nulidade de uma Árvore Geradora
- Árvores Geradoras Mínimas
 - Algoritmo de Prim
 - Algoritmo de Kruskal

* Consulte a bibliografia básica e complementar

Grafo Árvore

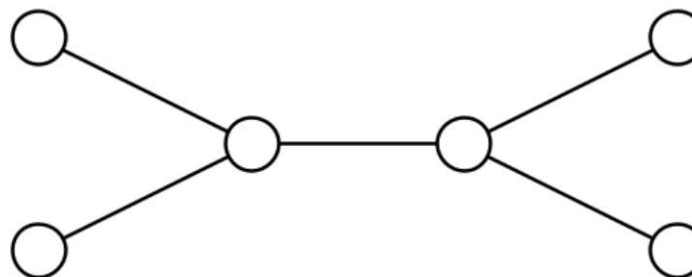
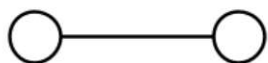
Distância entre pares de vértices

Excentricidade de um vértice

Centro, Raio e Diâmetro de uma Árvore

Grafo árvore

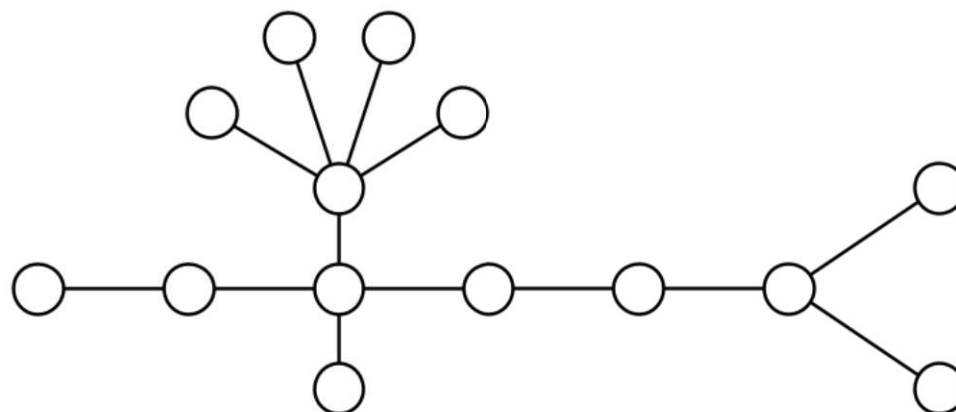
- Um grafo **árvore** é um grafo **simples** e **conexo** que **não contém circuitos** (caminho fechado)
- TEOREMA 1: Em uma árvore **existe um, e apenas um, caminho entre cada par de vértices** (v_i, v_j)
- TEOREMA 2: Se, em um grafo G existe **um único caminho entre qualquer par de vértices**, então G é um **grafo árvore**
- TEOREMA 3: Uma árvore com n vértices possui $n - 1$ arestas



Grafo árvore

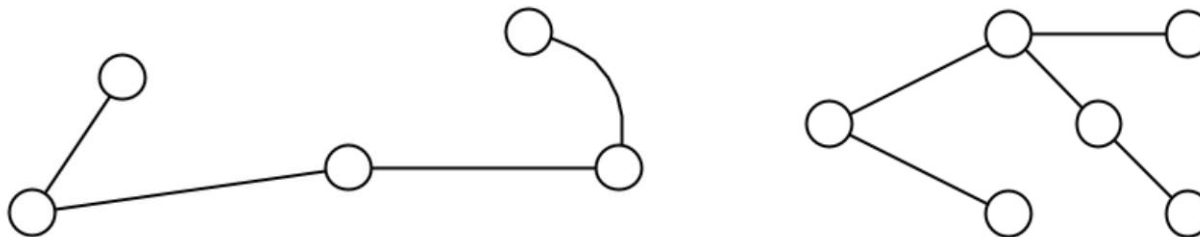
- TEOREMAS:

- Todo grafo G conexo com n vértices e $n - 1$ arestas é um grafo do tipo árvore
- Um grafo G é uma árvore se, e somente se, a remoção de qualquer aresta desconectar G , pois G é um grafo minimamente conectado
- Um grafo G conexo é uma árvore com n vértices e $n - 1$ arestas, onde não existem circuitos (caminho fechado)



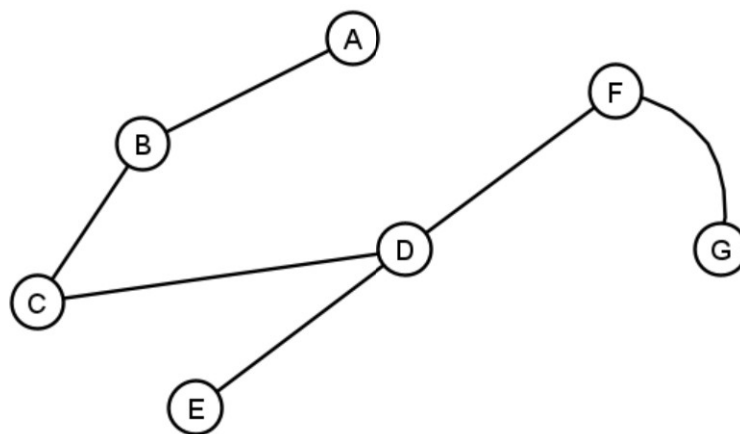
Grafo árvore simples

- Logo, um grafo G simples com n vértices é uma árvore, se:
 - G é conexo e sem circuito, ou
 - G é conexo e tem $n - 1$ arestas, ou
 - G não possui circuitos e tem $n - 1$ arestas, ou
 - Existe exatamente um caminho entre todos os pares de vértices (v_i, v_j) , ou
 - G é um grafo minimamente conectado/conexo
- Em qualquer árvore com pelo menos 2 vértices, existem pelo menos 2 vértices pendent



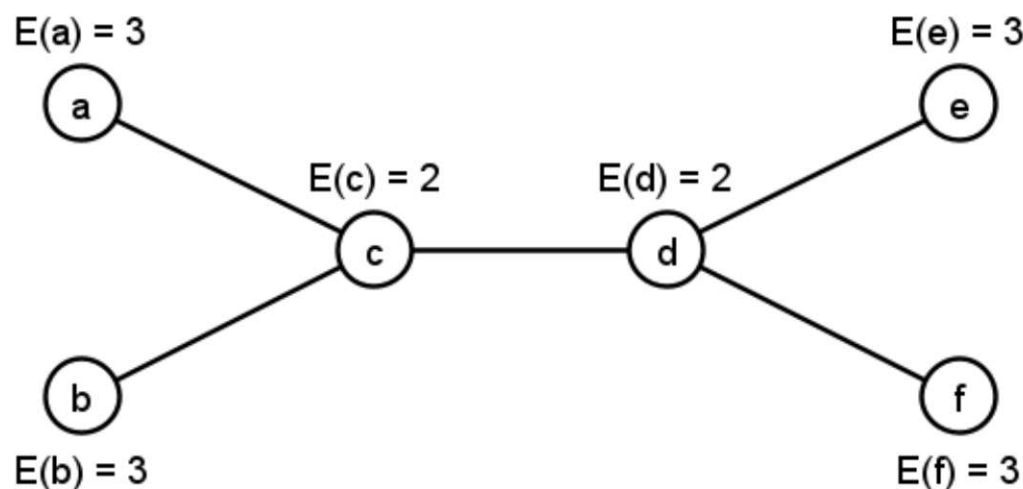
Distância entre vértices de um grafo árvore

- Em um grafo G , a **distância** d entre os pares v_i e v_j corresponde ao **número de arestas** do **menor caminho** que **conecta** v_i a v_j
- A **excentricidade** de um vértice v_i , em um grafo G **conexo**, é a **distância** de v_i ao **vértice mais distante** v_j de G
 - $E(v_i) = \max_{(v_i, v_j)} [d(v_i, v_j)]$



Excentricidade de um grafo árvore

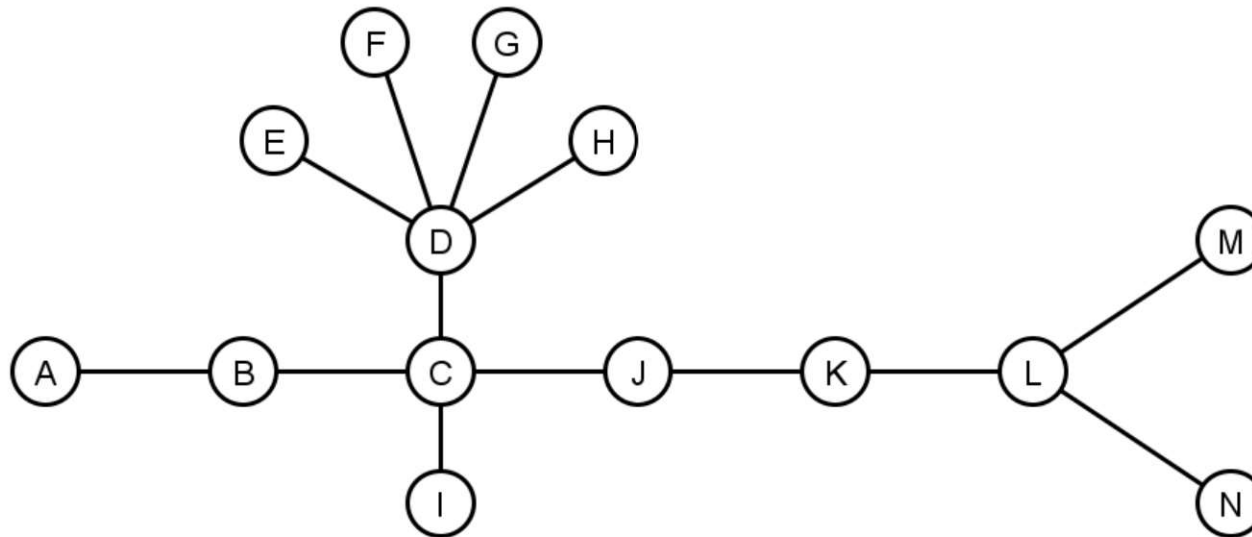
- O **centro** de um grafo G é o **vértice** v_i de **menor** **excentricidade** $E(v_i)$



- Qual é o **número máximo de centros** podem existir em um grafo **árvore**?
- Considere, hipoteticamente, um grafo **circuito**, quantos **centros** possui um grafo **circuito**?

Raio e Diâmetro de um grafo árvore

- TEOREMA: Toda **árvore** tem **exatamente 1 ou 2 centros**
- COROLÁRIO: Se uma **árvore** tem **2 centros**, então estes **centros são adjacentes**



- O **Raio** de um **árvore** é a **excentricidade do centro**
- O **Diâmetro** da **árvore** é o **comprimento do maior caminho em T**

Árvore Geradora

Galho de uma Árvore Geradora

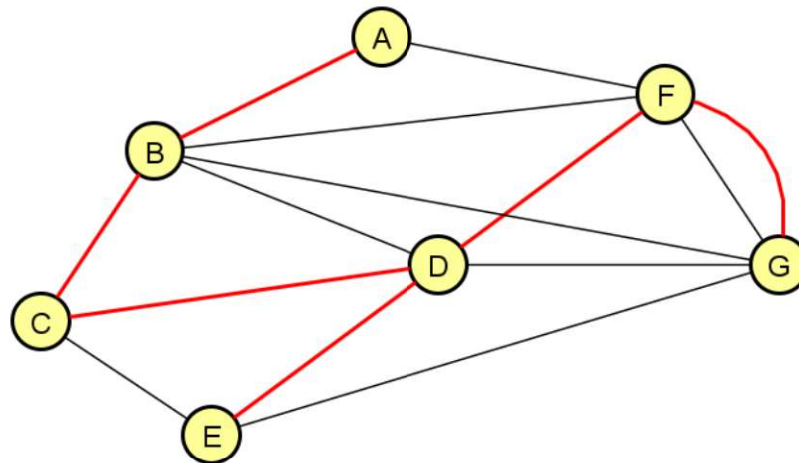
Corda de uma Árvore Geradora

Rank e Nulidade de Árvore Geradora

Algoritmos de Prim e Kruskal

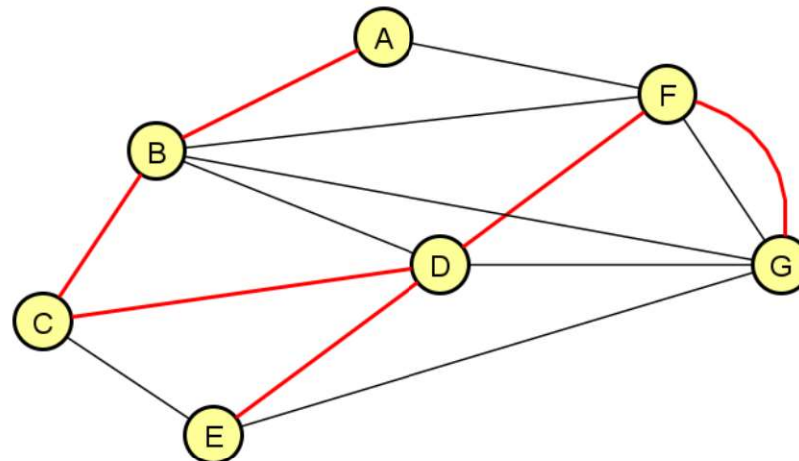
Árvore geradora

- A **árvore geradora** de um grafo G **conexo** é um **subgrafo** de G que contém **todos os vértices** de G formando uma **árvore**
- A **árvore geradora** somente é definida para um grafo **conexo**, pois **toda árvore é conexa**
- Um grafo G **não conexo** possui k **florestas geradoras**



Galho e Corda de uma árvore geradora

- Dado um grafo G e uma **árvore geradora** T de G , por definição:
 - **Galho**: É uma **aresta** de G em T
 - **Corda**: É uma **aresta** de G que **não pertence** a T
- TEOREMAS
 - Todo grafo G **conexo** tem uma **árvore geradora**
 - Seja G **conexo** com n vértices, e arestas, e uma árvore geradora T de G , então G tem $n - 1$ galhos e $e - n + 1$ cordas
 - Vértices: 7
 - Arestas: 14
 - Galhos: 6
 - Cordas: 8



Floresta geradora

- Definições:

- Rank de G é o número de galhos ($e \in T$) em qualquer floresta geradora k de G , sendo $n = V(G)$ de G

$$r = n - k$$

- Nulidade de G é o número de cordas em uma floresta geradora k de G , sendo $n = V(G)$, $e = E(G)$ e k componentes de G

$$\mu = e - n + k$$

- Rank + Nulidade = Número de arestas

$$r + \mu = e$$

Árvore Geradora Mínima (AGM)

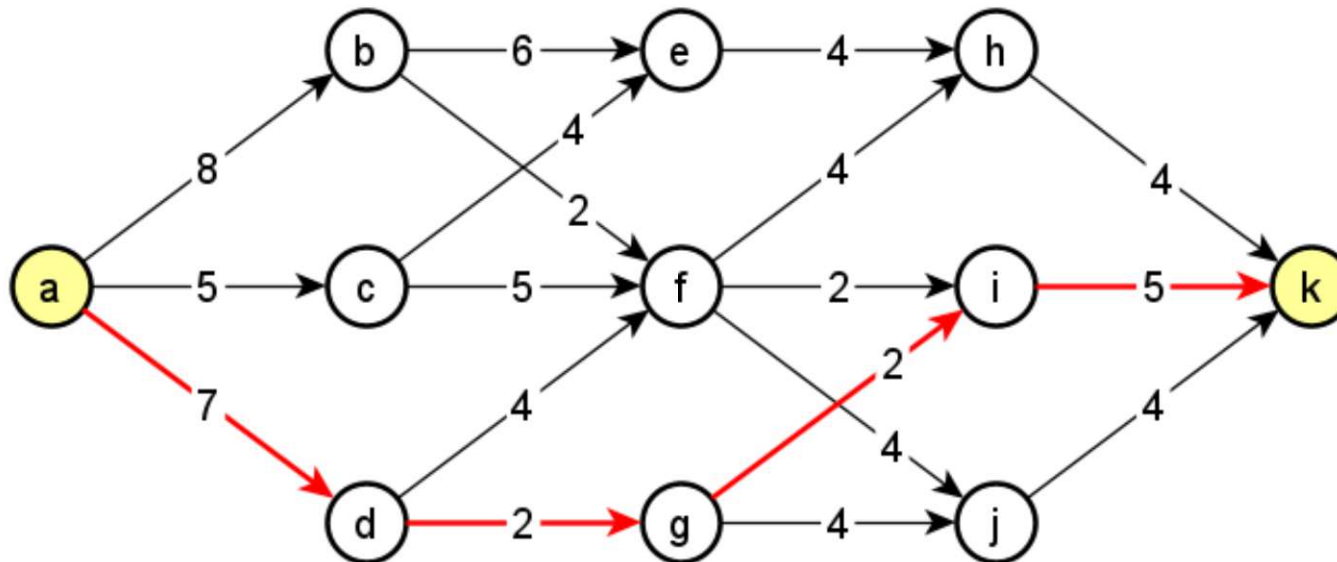
Árvore Geradora com menor peso

Algoritmo de Prim

Algoritmo de Kruskal

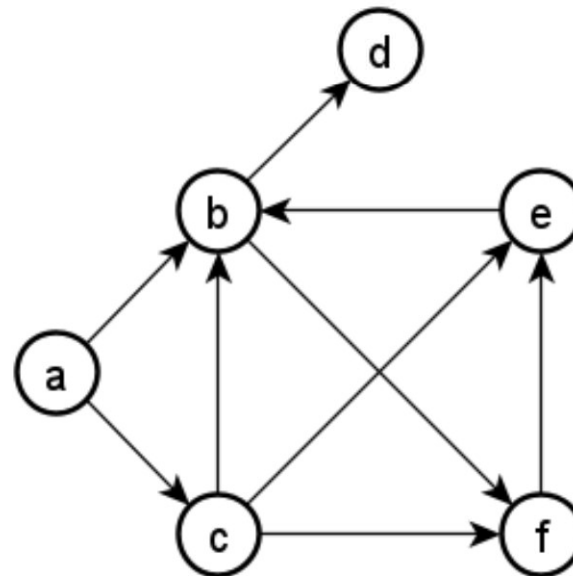
Árvore geradora mínima

- Árvore Geradora Mínima ou AGM é a árvore geradora de menor peso de G
 - A árvore geradora mínima é chamada de MST (*Minimum Spanning Tree*)
- Somente em um grafo G com pesos associados às arestas é possível encontrar uma árvore geradora mínima de G



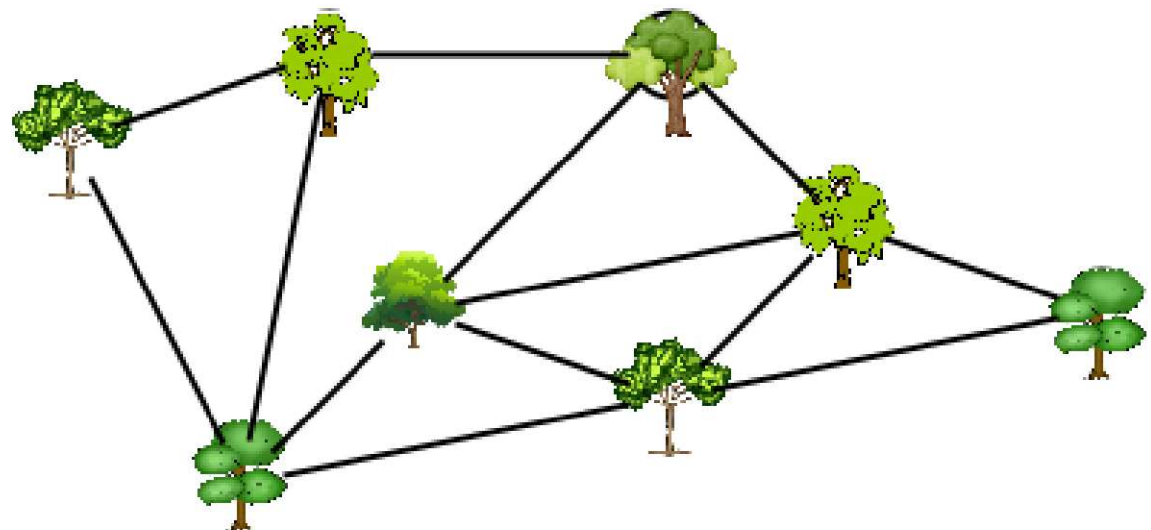
Problema do custo de estradas

- Devemos conectar n cidades $v_1, v_2, \dots, v_n$ através de uma rede de estradas
 - Considere c_{ij} como o custo de uma estrada entre as cidades v_i e v_j
 - A solução é encontrar a rede que minimiza o custo a fim de conectar as n cidades
- Modelagem
 - Vértices: Cidades
 - Arestas: Estradas com pesos



Problema do pomar

- Um pomar com 8 pés de frutas deve ser irrigado
 - Cada cano entre duas árvores tem um custo proporcional ao seu comprimento, esforço de implantação e fluxo de água
 - Como encontrar o sistema de irrigação ideal que minimiza o custo de implantação?
- Modelagem
 - Vértices: Árvores
 - Arestas: Canos com custos de implantação



Algoritmo de Prim (Pseudo código)

Prim(int v_i)

Inicialize todas as posições de p com -1

Insira 0 na posição i de v_i no vetor p

Insira v_i no vetor r

Insira todas as arestas incidentes de v_i no vetor w

while $p \subseteq -1$ **do**

Retire de w a aresta $E(v, x)$ de menor peso, sendo $x \notin p$

Marque a posição de $p[x]$ com o valor de v

Insira x no vetor r

Insira as arestas incidentes de x no vetor w , onde $x \notin p$

end while

end function

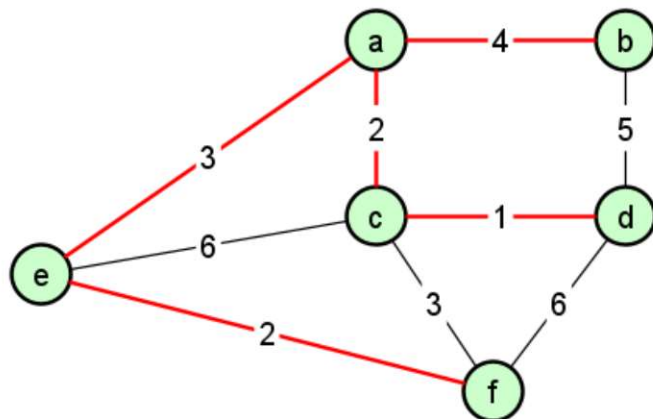
- Ordem de complexidade: $O(e + n \log n)$

Algoritmo de Prim

1) $r = \{\text{NULL}\}$
 $W = \{\text{NULL}\}$
 $P = \{0, -1, -1, -1, -1, -1\}$

2) $r = \{a\}$
 $W = \{4ab, 2ac, 3ae\}$
 $P = \{0, -1, a, -1, -1, -1\}$

3) $r = \{a, c\}$
 $W = 4ab, 3ae, 1cd, 6ce, 3cf$
 $P = \{0, -1, a, c, -1, -1\}$



4) $r = \{a, c, d\}$
 $W = \{4ab, 3ae, 6ce, 3cf, 5db, 6df\}$
 $P = \{0, -1, a, c, a, -1\}$

5) $r = \{a, c, d, e\}$
 $W = \{4ab, 3ef, 5db, 6df, 2ef\}$
 $P = \{0, -1, a, c, a, e\}$

6) $r = \{a, c, d, e, f\}$
 $W = \{4ab, 5db\}$
 $P = \{0, a, a, c, a, e\}$

7) $r = \{a, c, d, e, f, b\}$
 $W = \{\text{NULL}\}$
 $P = \{0, a, a, c, a, e\}$

Algoritmo de Kruskal (Pseudo código)

Kruskal(int v_i)

Inicialize todas as posições de p com -1

Inicialize todas as posições de T com i de cada v_i

Insira 0 na posição i de v_i no vetor p

Insira todas as arestas incidentes de v_i no vetor w , ordene w

while $p \subseteq -1$ **and** $!UniqueValue(T)$ **do**

 Retire de w a aresta $E(v, x)$ de menor peso, sendo $x \notin p$

 *Aresta $E(v, x)$ retirada de w , não pode formar circuito

 Marque a posição de $p[x]$ com o valor de v

 Troque os valores iguais de v em T pelo valor de x

 Insira as arestas incidentes de x no vetor w , e ordene w

end while

end function

- Ordem de complexidade: $O(e \log n)$

Algoritmo de Kruskal

1) $r = \{\text{NULL}\}$

$W = \{\text{NULL}\}$

$P = \{0, -1, -1, -1, -1, -1\}$

$T = \{0, 1, 2, 3, 4, 5\}$

2) $r = \{a\}$

$W = \{5ab, 10ad\}$

$P = \{0, a, -1, -1, -1, -1\}$

$T = \{1, 1, 2, 3, 4, 5\}$

3) $r = \{a, b\}$

$W = \{5bc, 10ad, 10bd, 20be\}$

$P = \{0, a, b, -1, -1, -1\}$

$T = \{2, 2, 2, 3, 4, 5\}$

4) $r = \{a, b, c\}$

$W = \{10ad, 10bd, 20be, 20ed, 30cf\}$

$P = \{0, a, b, a, -1, -1\}$

$T = \{3, 3, 3, 3, 4, 5\}$

5) $r = \{a, b, c, d\}$

$W = \{20be, 20de, 30cf\}$

$P = \{0, a, b, a, b, -1\}$

$T = \{4, 4, 4, 4, 4, 5\}$

6) $r = \{a, b, c, d, e\}$

$W = \{10ef, 30ef\}$

$P = \{0, a, b, a, b, e\}$

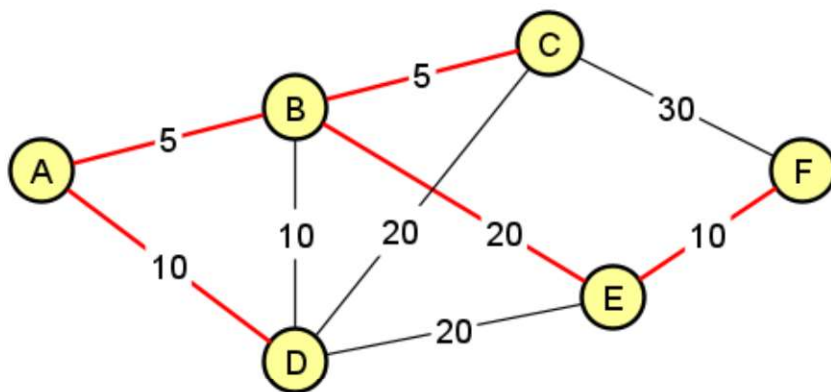
$T = \{5, 5, 5, 5, 5, 5\}$

7) $r = \{a, b, c, d, e, f\}$

$W = \{\text{NULL}\}$

$P = \{0, a, b, a, b, e\}$

$T = \{5, 5, 5, 5, 5, 5\}$



Próxima aula...

- Atividades e compromissos:
 - Completar a modelagem dos problemas apresentados!
 - Implementar os algoritmos de Teoria de Grafos!

Até o nosso próxima encontro...